# OsciMobile

*Mobile-Based Smart Oscilloscope using ESP32*

Wireless Real-Time Waveform Analyzer with Web Dashboard


| Name | Roll Number |
| --- | --- |
| Rohit Kumar Sahu | 123EE0329 |



**April 2026** NIT Rourkela, Odisha, India

## Contents

**Abstract** **2**

**1 Name of the Product** **2**

**2 Specifications** **2**

2.1 Microcontroller & Processing . . . . . 2

2.2 Sensing Parameters . . . . . 3

2.3 Power Supply . . . . . 3

**3 Application** **3**

**4 Certifications Required** **4**

**5 Component List** **4**

**6 Photo of Product with Component Marking** **5**

**7 Description of Different Parts of the Product** **6**

7.1 Hardware Pin Mapping (ESP32) . . . . . 6

7.2 Sensor Sub-Circuits . . . . . 6

7.2.1 DC Voltage Sensing Circuit (LMH6644 Op-Amp Buffer) . . . . . 6

7.2.2 AC Voltage Sensing (ZMPT101B) . . . . . 6

7.2.3 Current Sensing (ACS712-20A) . . . . . 7

7.3 OsciDash Web Dashboard - Features . . . . . 7

7.4 Firmware Architecture (FreeRTOS Dual-Core) . . . . . 8

7.5 Firmware Source Code . . . . . 8

**8 Difficulty While Developing the Product** **13**

**9 Scope of Improvement** **14**

**10 List of Available Similar Products in the Market** **14**

**11 Costing** **15**

**12 Marketing Strategy** **15**

12.1 Target Markets . . . . . 15

12.2 Growth Strategy . . . . . 15

12.3 Revenue Projections (Year 1) . . . . . 16

## Abstract

**OsciMobile** is a portable, wireless, and high-performance smart oscilloscope designed for real-time waveform analysis using a smartphone or PC as its display. By harnessing the dual-core ESP32 microcontroller and high-speed WebSocket communication, this device eliminates the need for expensive dedicated hardware screens. The project integrates multi-channel sensing - AC voltage (0–250V via ZMPT101B), DC voltage (0–21V via a precision resistor-divider with LMH6644 op-amp buffer), and bidirectional current (±20A via ACS712) - and transmits processed data wirelessly to **OsciDash**, a professional web-based oscilloscope dashboard. The result is a professional-grade "Internet of Instruments" (IoI) tool that delivers laboratory-level measurements (RMS, Power, Frequency, Peak) in a compact, sub-5000 prototype form factor, making it ideal for educational labs, field diagnostics, and IoT energy monitoring.


## 1 Name of the Product

**OsciMobile**

*A high-performance, mobile-based smart oscilloscope and waveform analyzer powered by the ESP32 microcontroller*

The name *Osci* (oscilloscope) combined with *Mobile* (smartphone-driven) reflects the core innovation: turning any Wi-Fi-enabled screen into a laboratory-grade oscilloscope. The device streams live sensor data via WebSocket to the OsciDash web interface, accessible from any browser without any app installation.

## 2 Specifications

### 2.1 Microcontroller & Processing

| Parameter | Value / Detail |
|---|---|
| Chipset | ESP32-WROOM-32, Dual-Core Xtensa LX6 @ 240 MHz |
| Flash / SRAM | 4 MB Flash, 520 KB SRAM |
| ADC | 12-bit SAR ADC, effective resolution ≈10-bit due to noise |
| Sampling Rate | 10 kSPS (configurable via `SAMPLE_INTERVAL_US`) |
| Buffer Size | 256 samples per frame |
| Communication | 2.4 GHz Wi-Fi 802.11 b/g/n; WebSocket (full-duplex TCP) |
| Display | 2.8" TFT ILI9341 via Hardware SPI + OsciDash Web UI |
| RTOS | FreeRTOS (dual-core task pinning) |

Table 1: Microcontroller and processing specifications

### 2.2 Sensing Parameters

| Channel | Sensor | Range | Circuit Details |
|---|---|---|---|
| DC Voltage | LMH6644 + Resistor Divider | 0–21 V DC | 5.6 kΩ/1 kΩ divider; LMH6644 unity-gain buffer on GPIO 36 |
| AC Voltage | ZMPT101B Transformer | 0–250 V AC | Isolated transformer with trimmer for calibration; output on GPIO 34 |
| Current | ACS712-20A (Hall Effect) | ±20 A | 185 mV/A sensitivity; zero-current offset = 2.5 V; output on GPIO 33 |

Table 2: Sensing channels and their hardware details

### 2.3 Power Supply

- Input: 230 V AC mains via SMPS (Mean Well LRS-50-36)
- Regulated to 5 V DC via LM2596 Buck Converter for logic and sensors
- ESP32 operates at 3.3 V (internal LDO on DevKit board)

## 3 Application

1. **Mobile-Based Oscilloscope Learning:** Enables students to visualize real-time waveforms on mobile or laptop browsers using ESP32, making signal analysis accessible without traditional bulky equipment.

2. **Portable Measurement Tool:** Provides a compact, wireless solution for measuring voltage, current, and waveform signals in real-world environments without requiring dedicated lab instruments.

3. **Real-Time Web Monitoring:** Allows live transmission of electrical parameters (voltage, current, power) from ESP32 to a web dashboard using UDP and WebSocket communication.

4. **Low-Cost Smart Instrumentation:** Acts as an affordable alternative to basic oscilloscopes, suitable for students, hobbyists, and small-scale experimental setups.

5. **Wireless Data Visualization:** Eliminates wired connections by enabling data viewing directly on a browser, supporting flexibility in testing and monitoring electrical signals remotely.

## 4 Certifications Required

| Certification | Jurisdiction | Purpose |
|---|---|---|
| BIS (IS 13252) | India | Safety and performance compliance for consumer electronics sold in India |
| IEC 61010-1 | International | Safety requirements for electrical equipment used for measurement |
| CE Marking | European Union | Conformance with EU health, safety, and environmental standards |
| RoHS Directive | EU / Global | Restriction of hazardous substances in electronic equipment |
| FCC Part 15 (Class B) | USA | Electromagnetic interference limits for Wi-Fi module (ESP32) |
| WPC / TEC Approval | India | Approval for use of 2.4 GHz Wi-Fi radio module in India |

Table 3: Certifications required for commercial deployment

## 5 Component List

| # | Item Name | Specifications | Qty | Est. Cost |
|---|---|---|---|---|
| 1 | ESP32 DevKit V1 | WROOM-32, 30-pin, 240 MHz Dual-Core | 1 | 380 |
| 2 | Op-Amp IC | LMH6644 Rail-to-Rail, SOT-23 package | 1 | 85 |
| 3 | Current Sensor | ACS712-20A Hall-Effect Module | 1 | 120 |
| 4 | AC Voltage Sensor | ZMPT101B Isolated Transformer Module | 1 | 150 |
| 5 | TFT Display | 2.4” ILI9341 SPI, 240×320 px | 1 | 320 |
| 6 | Buck Converter | LM2596 Step-down, 5V/3A Output | 1 | 90 |
| 7 | Resistors | 1 kΩ, 5.6 kΩ, 20 kΩ (1% precision) | 10 | 20 |
| 8 | SPDT Switches | Toggle Switches, 2A rated | 3 | 30 |
| 9 | Header Pins | 2.54 mm Male/Female, 40-pin strips | 4 | 40 |
| 10 | Veroboard | 9×15 cm Copper Strip Board | 1 | 60 |
| 11 | Connecting Wires | Jumper wires + custom wiring (22 AWG) | – | 80 |
| 12 | Crocodile Clips | Test probe clips (Red & Black pair) | 2 | 40 |
| 13 | SMPS Power Supply | Mean Well LRS-50-36, 230V AC to 36V DC | 1 | 500 |
| | | | **Total BOM Cost** | **1,915** |

Table 4: Complete Bill of Materials with estimated costs

## 6 Photo of Product with Component Marking

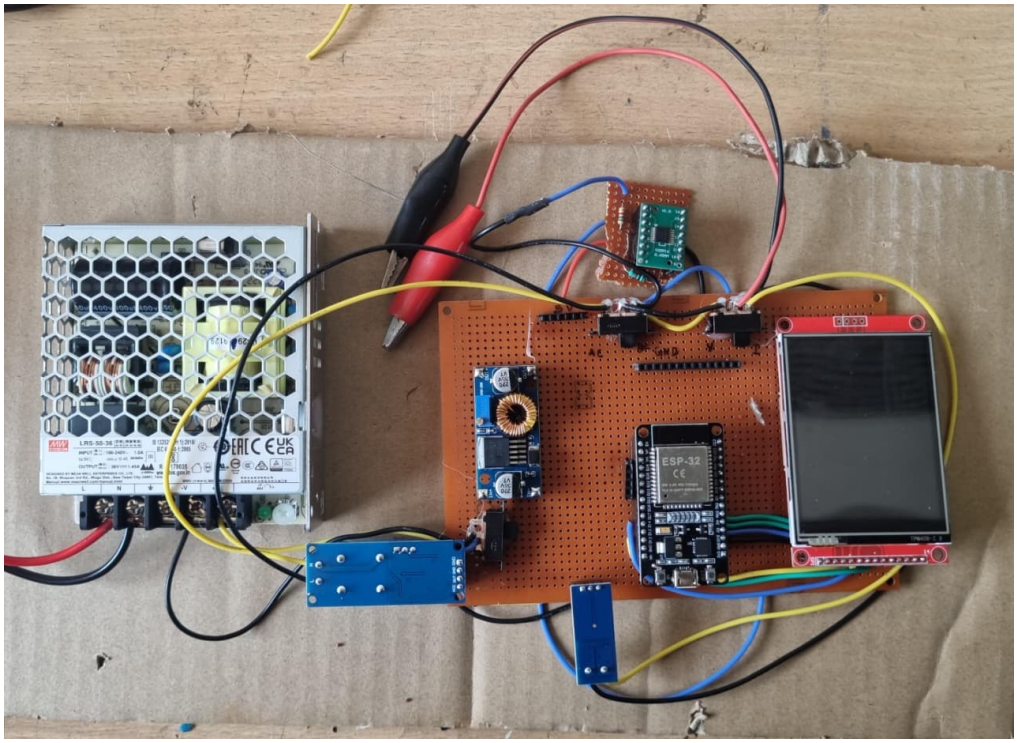

Figure 1: Hardware prototype of OsciMobile assembled on Veroboard. Visible components include (left to right): Mean Well SMPS, LM2596 Buck Converter, ACS712 Current Sensor (blue module at bottom), LMH6644 DC Voltage Sensor circuit (top-right daughter board), ESP32-WROOM DevKit (centre), and ILI9341 TFT Display (far right).

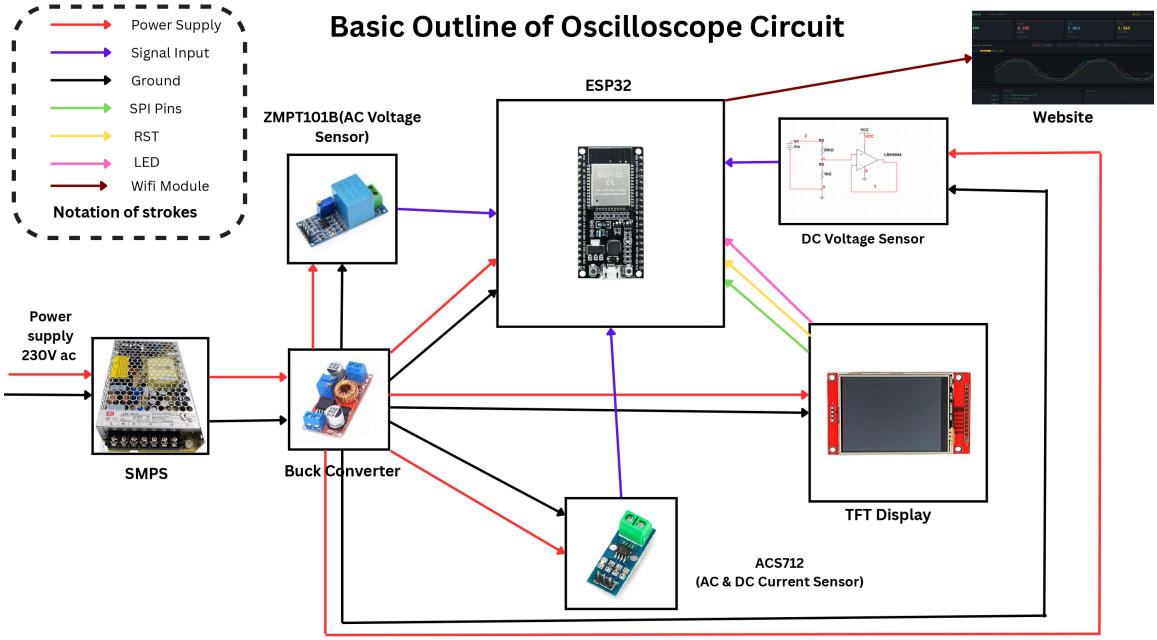


Figure 2: Basic outline of the OsciMobile oscilloscope circuit showing signal flow: 230V AC SMPS → Buck Converter → ZMPT101B (AC Voltage), ACS712 (Current), and LMH6644 (DC Voltage) sensors feeding into the ESP32. The ESP32 drives the TFT Display via SPI and transmits data to the website via Wi-Fi.

# 7 Description of Different Parts of the Product

## 7.1 Hardware Pin Mapping (ESP32)

| GPIO Pin | Signal Name | Connected To | Description |
|---|---|---|---|
| **Analog Input Channels (ADC1 - Safe with Wi-Fi)** | | | |
| GPIO 34 | AC_V_IN | ZMPT101B Output | AC voltage waveform input (ADC1_CH6, input only) |
| GPIO 33 | CURR_IN | ACS712 Output | Current waveform input (ADC1_CH5) |
| GPIO 32 | DC_V_IN | LMH6644 Buffer Out | DC voltage input via resistor divider (VP pin, input only) |
| **SPI Bus - TFT ILI9341 Display** | | | |
| GPIO 13 | TFT_MOSI | TFT SDI/MOSI | SPI Master Out - pixel data to display |
| GPIO 12 | TFT_MISO | TFT SDO/MISO | SPI Master In (not used for write-only TFT) |
| GPIO 14 | TFT_SCLK | TFT SCK | SPI Clock |
| GPIO 15 | TFT_CS | TFT CS | Chip Select (active LOW) |
| GPIO 4 | TFT_DC | TFT D/C | Data/Command select (HIGH=data, LOW=cmd) |
| GPIO 2 | TFT_RST | TFT RESET | Hardware reset for display initialization |
| **Power Pins** | | | |
| 3.3V | VCC | Sensors, LMH6644 | 3.3V logic supply from DevKit onboard LDO |
| 5V (VIN) | 5V Rail | ACS712, ZMPT, TFT | 5V supply from LM2596 Buck Converter |
| GND | GND | All modules | Common ground reference |

Table 5: Complete ESP32 pin mapping for OsciMobile

## 7.2 Sensor Sub-Circuits

### 7.2.1 DC Voltage Sensing Circuit (LMH6644 Op-Amp Buffer)

The DC voltage input (0–21 V) is scaled down to the ESP32's ADC range (0–3.3 V) using a precision voltage divider:

$$V_{\text{out}} = V_{\text{in}} \times \frac{R_2}{R_1 + R_2} = V_{\text{in}} \times \frac{1\,\text{k}\Omega}{20\,\text{k}\Omega + 1\,\text{k}\Omega} \approx V_{\text{in}} \times 0.1515$$

The LMH6644 rail-to-rail op-amp is configured in unity-gain (voltage follower) mode to provide a low-impedance drive to GPIO 36, preventing ADC loading effects and improving measurement accuracy.

### 7.2.2 AC Voltage Sensing (ZMPT101B)

The ZMPT101B is a current transformer-based voltage sensor module with a built-in op-amp for signal conditioning. It provides galvanic isolation from the mains (up to 1000 V isolation) and scales the 50 Hz AC waveform to a centred 2.5 V DC-offset signal readable by the ESP32 ADC on GPIO 34.

### 7.2.3 Current Sensing (ACS712-20A)

The ACS712 uses the Hall Effect to measure current flowing through its copper conductor without breaking the circuit. Its output is:

$$V_{\text{out}} = 2.5\,\text{V} + \text{Sensitivity} \times I \quad (\text{Sensitivity} = 185\,\text{mV/A for 20A variant})$$

At zero current: $V_{\text{out}} = 2.5\,\text{V}$. This maps to `ADC_CENTER = 2048` in firmware.

## 7.3 OsciDash Web Dashboard - Features

Figure 3: OsciDash Web Interface - professional responsive dashboard for real-time waveform visualization, metric tracking, and data export.

The **OsciDash** web interface is a fully browser-based, responsive professional oscilloscope dashboard. It communicates with the ESP32 over WebSocket (port 8888) and provides the following features:

1. **Real-Time Waveform Visualization:** Oscilloscope-style dual-axis graph displays Voltage (green) and Current (red) waveforms simultaneously with near-zero latency via WebSocket streaming.
2. **Live Metric Display:** Four header cards show live values - Voltage (V DC), Current (A), Power (W), and V-RMS - each with peak and sample count sub-values.
3. **Adjustable Trigger Level:** An interactive slider sets the trigger voltage (e.g., 2.50 V) for stable waveform display, shown as a dashed horizontal line on the graph.
4. **Zoom & Auto-Scale:** X-axis and Y-axis zoom controls (+ and − buttons) plus an AUTO button for automatic scaling to fit the captured waveform.
5. **Freeze / Record:** FREEZE pauses the live display for detailed inspection. REC continuously records incoming frames to an internal buffer.
6. **CSV Export:** Exports the captured waveform buffer directly as a `.csv` file for offline analysis in Excel or Python.

7. **Frequency Detection:** Zero-crossing algorithm detects fundamental frequency of AC waveforms in real-time.
8. **Peak Value Tracking & Statistics:** Tracks $V_{min}$, $V_{max}$, $I_{min}$, $I_{max}$, and $P_{max}$ continuously, shown in the Statistics panel.
9. **Event Logging:** Timestamped log of WebSocket connection events, errors, and reconnections displayed in the Event Log panel.
10. **Fault Alert System:** Visual fault indicators appear in the Fault Alerts panel if measured voltage or current exceeds safe thresholds.
11. **DEMO / LIVE Mode:** Automatic fallback to demo mode (simulated sine wave) when the ESP32 is offline; switches to LIVE when connected.
12. **Connection Status Monitor:** Real-time ONLINE/OFFLINE indicator with auto-reconnect logic (retries every 3 seconds).

### 7.4 Firmware Architecture (FreeRTOS Dual-Core)

The ESP32 firmware uses FreeRTOS task pinning to separate time-critical acquisition from network I/O:

- **Core 1 - Task_Acquisition_and_Display:** Runs the 10 kSPS sampling loop, computes RMS/DC values, and renders the waveform on the TFT display.
- **Core 0 - Task_Network_UDP:** Monitors the `bufferReadyForNetwork` flag and transmits JSON payloads via UDP. Waveform data is sent every frame; metrics are sent every 10th frame to conserve bandwidth.

### 7.5 Firmware Source Code

```
#include <WiFi.h>
#include <WiFiUdp.h>
#include <SPI.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ILI9341.h>
#include <math.h>
#include "esp_adc_cal.h"

// --- Network ---
const char* ssid = "tiburon";
const char* password = "teamtiburon";
const char* udpAddress = "10.175.11.105";
const int udpPort = 8888;

WiFiUDP udp;

// --- TFT PINS ---
#define TFT_MISO 12
#define TFT_MOSI 13
#define TFT_SCLK 14
```

```
#define TFT_CS   15
#define TFT_DC   4
#define TFT_RST  2

SPIClass hspi(HSPI);
Adafruit_ILI9341 tft = Adafruit_ILI9341(&hspi, TFT_DC, TFT_CS, TFT_RST);

// --- SENSOR PINS ---
#define PIN_ZMPT 34
#define PIN_DC   32
#define PIN_ACS  33

#define SCREEN_W 320
#define SCREEN_H 240
#define ROTATION 1

#define BUFFER_SIZE 200
#define SAMPLE_INTERVAL_US 100

// --- CALIBRATION ---
float CAL_DC_VOLTS = 6.6;
float CAL_AC_VOLTS = 600;
float CAL_CURRENT  = 1.0;

// --- BUFFERS ---
volatile int bufferZMPT[BUFFER_SIZE];
volatile int bufferDC[BUFFER_SIZE];
volatile int bufferACS[BUFFER_SIZE];

volatile float acVoltsRMS = 0.0;
volatile float dcVolts = 0.0;
volatile float currentRMS = 0.0;

volatile bool bufferReadyForNetwork = false;

esp_adc_cal_characteristics_t adc1_chars;

TaskHandle_t TaskAcquisition;
TaskHandle_t TaskNetwork;

// ==========================================================
// CORE 1: ACQUISITION + DISPLAY
// ==========================================================
void Task_Acquisition_and_Display(void *pvParameters) {

  for (;;) {

    long sumZMPT = 0;
    long sumDC   = 0;
    long sumACS  = 0;

    uint32_t nextSampleTime = micros();
```

```

    for (int i = 0; i < BUFFER_SIZE; i++) {

      int rawZMPT = analogRead(PIN_ZMPT);
      int rawDC   = analogRead(PIN_DC);
      int rawACS  = analogRead(PIN_ACS);

      uint32_t mvZMPT = esp_adc_cal_raw_to_voltage(rawZMPT, &adc1_chars);
      uint32_t mvDC   = esp_adc_cal_raw_to_voltage(rawDC, &adc1_chars);
      uint32_t mvACS  = esp_adc_cal_raw_to_voltage(rawACS, &adc1_chars);

      bufferZMPT[i] = mvZMPT;
      bufferDC[i]   = mvDC;
      bufferACS[i]  = mvACS;

      sumZMPT += mvZMPT;
      sumDC   += mvDC;
      sumACS  += mvACS;

      nextSampleTime += SAMPLE_INTERVAL_US;
      while (micros() < nextSampleTime);
    }

    // --- OFFSET ---
    float meanZMPT = sumZMPT / (float)BUFFER_SIZE;
    float meanDC   = sumDC   / (float)BUFFER_SIZE;
    float meanACS  = sumACS  / (float)BUFFER_SIZE;

    float sqSumZMPT = 0;
    float sqSumACS  = 0;

    for (int i = 0; i < BUFFER_SIZE; i++) {

      float centeredZMPT = bufferZMPT[i] - meanZMPT;
      sqSumZMPT += centeredZMPT * centeredZMPT;

      float centeredACS = bufferACS[i] - meanACS;
      sqSumACS += centeredACS * centeredACS;
    }

    // --- RMS ---
    float rmsRaw = sqrt(sqSumZMPT / BUFFER_SIZE) / 1000.0;
    acVoltsRMS = (rmsRaw < 0.02) ? 0 : rmsRaw * CAL_AC_VOLTS;

    dcVolts = (meanDC / 1000.0) * CAL_DC_VOLTS;

    float rmsC = sqrt(sqSumACS / BUFFER_SIZE) / 1000.0;
    currentRMS = (rmsC < 0.01) ? 0 : rmsC * CAL_CURRENT;

    bufferReadyForNetwork = true;

    // ========================================================
```

```
    // DISPLAY (3 ZONES SETUP)
    // ======================================================
    int zH = SCREEN_H / 3;
    tft.setTextSize(2);

    // --- ZONE 1: AC TEXT & CLEAR ---
    tft.setCursor(5, 5);
    tft.setTextColor(ILI9341_GREEN, ILI9341_BLACK);
    tft.print("AC: "); tft.print(acVoltsRMS, 2); tft.println(" V   ");

    int z1_top = 25;
    int z1_bot = zH - 5;
    tft.fillRect(0, z1_top, SCREEN_W, z1_bot - z1_top, ILI9341_BLACK);

    // --- ZONE 2: DC TEXT & CLEAR ---
    tft.setCursor(5, zH + 5);
    tft.setTextColor(ILI9341_YELLOW, ILI9341_BLACK);
    tft.print("DC: "); tft.print(dcVolts, 2); tft.println(" V   ");

    int z2_top = zH + 25;
    int z2_bot = (zH * 2) - 5;
    tft.fillRect(0, z2_top, SCREEN_W, z2_bot - z2_top, ILI9341_BLACK);

    // --- ZONE 3: CURRENT TEXT & CLEAR ---
    tft.setCursor(5, (zH * 2) + 5);
    tft.setTextColor(ILI9341_CYAN, ILI9341_BLACK);
    tft.print("Cur: "); tft.print(currentRMS, 2); tft.println(" A   ");

    int z3_top = (zH * 2) + 25;
    int z3_bot = SCREEN_H - 5;
    tft.fillRect(0, z3_top, SCREEN_W, z3_bot - z3_top, ILI9341_BLACK);

    // ======================================================
    //        DRAW 3 SEPARATE WAVEFORMS
    // ======================================================
    for (int i = 0; i < BUFFER_SIZE - 1; i++) {

      int x1 = map(i, 0, BUFFER_SIZE, 0, SCREEN_W);
      int x2 = map(i+1, 0, BUFFER_SIZE, 0, SCREEN_W);

      // --- 1. AC WAVEFORM (Zoomed: -1000 to 1000 mV) ---
      int ac1 = bufferZMPT[i] - meanZMPT;
      int ac2 = bufferZMPT[i+1] - meanZMPT;
      int y1_ac = map(ac1, -1000, 1000, z1_bot, z1_top);
      int y2_ac = map(ac2, -1000, 1000, z1_bot, z1_top);
      tft.drawLine(x1, y1_ac, x2, y2_ac, ILI9341_GREEN);

      // --- 2. DC WAVEFORM (Absolute: 0 to 3300 mV) ---
      int y1_dc = map(bufferDC[i], 0, 3300, z2_bot, z2_top);
      int y2_dc = map(bufferDC[i+1], 0, 3300, z2_bot, z2_top);
      tft.drawLine(x1, y1_dc, x2, y2_dc, ILI9341_YELLOW);

```

```
      // --- 3. CURRENT WAVEFORM (Zoomed: -1000 to 1000 mV) ---
      int cur1 = bufferACS[i] - meanACS;
      int cur2 = bufferACS[i+1] - meanACS;
      int y1_cur = map(cur1, -1000, 1000, z3_bot, z3_top);
      int y2_cur = map(cur2, -1000, 1000, z3_bot, z3_top);
      tft.drawLine(x1, y1_cur, x2, y2_cur, ILI9341_CYAN);
    }

    vTaskDelay(pdMS_TO_TICKS(5));
  }
}

// ============================================================
// NETWORK
// ============================================================
void Task_Network_UDP(void *pvParameters) {

  for (;;) {

    if (WiFi.status() != WL_CONNECTED) {
      WiFi.reconnect();
    }

    if (bufferReadyForNetwork) {

     String payload = "{";
payload += "\"rms\":" + String(acVoltsRMS) + ",";
payload += "\"current\":" + String(currentRMS) + ",";
payload += "\"dc_v\":" + String(dcVolts);
payload += "}";

      udp.beginPacket(udpAddress, udpPort);
      udp.print(payload);
      udp.endPacket();

      bufferReadyForNetwork = false;
    }

    vTaskDelay(pdMS_TO_TICKS(5));
  }
}

// ============================================================
// SETUP
// ============================================================
void setup() {

  Serial.begin(115200);

  esp_adc_cal_characterize(ADC_UNIT_1, ADC_ATTEN_DB_11, ADC_WIDTH_BIT_12, 1100, &
    adc1_chars);

```

```
  analogReadResolution(12);
  analogSetPinAttenuation(PIN_ZMPT, ADC_11db);
  analogSetPinAttenuation(PIN_ACS,  ADC_11db);
  analogSetPinAttenuation(PIN_DC,   ADC_11db);

  hspi.begin(TFT_SCLK, TFT_MISO, TFT_MOSI, TFT_CS);

  tft.begin();
  tft.setRotation(ROTATION);
  tft.fillScreen(ILI9341_BLACK);

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }

  xTaskCreatePinnedToCore(Task_Acquisition_and_Display, "Task_Acq", 10000, NULL,
    1, &TaskAcquisition, 1);
  xTaskCreatePinnedToCore(Task_Network_UDP, "Task_Net", 10000, NULL, 1, &
    TaskNetwork, 0);
}

void loop() {
  vTaskDelete(NULL);
}
```

Listing 1: OsciMobile ESP32 Firmware - Full Source (oscimobile.ino)

## 8 Difficulty While Developing the Product

1. **ESP32 ADC Non-Linearity:** The ESP32's internal SAR ADC exhibits significant non-linearity near the supply rails (readings below ∼0.1 V and above ∼3.1 V are unreliable). This caused distorted AC waveform peaks. *Resolution:* Software calibration using a look-up table (LUT) and restricting the signal swing to the 0.15–3.1 V linear region via sensor gain trimming.

2. **Wi-Fi RF Noise on High-Impedance Inputs:** The ESP32's 2.4 GHz Wi-Fi transceiver introduced RF interference into the high-impedance LMH6644 op-amp inputs, causing high-frequency spikes in DC voltage measurements. *Resolution:* Added RC low-pass filters (100 Ω + 100 nF) at each ADC input pin. Ground plane stitching on the Veroboard also helped.

3. **ADC1 vs ADC2 Conflict with Wi-Fi:** The ESP32's ADC2 cannot be used when Wi-Fi is active (silicon limitation). All sensing pins were therefore carefully mapped to ADC1 channels (GPIO 32–39) only. *Resolution:* Deliberate use of GPIO 34, 33, and 36 - all on ADC1.

4. **UDP Packet Size vs JSON String Length:** Sending a 256-sample waveform as a JSON string approached the UDP payload limit. *Resolution:* Manual JSON string construction (avoiding ArduinoJson overhead) and keeping `BUFFER_SIZE` $\leq 256$ to fit within a single UDP datagram.

5. **TFT Display Flicker:** Full-screen `fillScreen(TFT_BLACK)` before each redraw caused visible flickering. *Resolution:* Considered double-buffering; partially mitigated by reducing `vTaskDelay`

and drawing only changed regions where possible.

## 9 Scope of Improvement

1. **Custom PCB Design:** Designing a custom 2-layer PCB to replace the veroboard prototype, reducing noise, improving signal integrity, and enabling compact miniaturized deployment.

2. **External High-Resolution ADC:** Adding a dedicated external 12-bit or 16-bit ADC (e.g., ADS1115, ADS8688, ADS8320, MCP3204) to overcome ESP32 ADC non-linearity limitations and achieve higher accuracy and bandwidth.

3. **WebSocket Protocol Upgrade:** Replacing UDP with WebSocket directly on the ESP32 (using the `AsyncWebServer` library) to eliminate the Node.js relay server and improve latency.

4. **Bluetooth Low Energy (BLE) Connectivity:** Integrating BLE communication for direct mobile connectivity without requiring Wi-Fi infrastructure, enabling flexible deployment.

5. **Battery-Powered Operation:** Incorporating a rechargeable LiPo battery with battery management system (BMS) for fully portable, wireless field operation.

6. **Multi-Channel Measurement:** Enabling 4-channel simultaneous measurement using an external analog multiplexer (e.g., CD4051) for expanded signal acquisition capability.

7. **Touchscreen Interface:** Adding a touchscreen TFT display (ILI9341 with XPT2046 controller) for direct on-device waveform interaction and control.

8. **Differential Measurement Inputs:** Providing galvanically isolated differential inputs for safe measurement of floating or high-side signals in complex circuits.

9. **Enclosure and Mechanical Design:** Designing a 3D-printed enclosure with proper strain relief and IP-rated connectors for rugged and safe field usage.

## 10 List of Available Similar Products in the Market

| Product | Price (Approx.) | Bandwidth | Key Limitation vs. OsciMobile |
|---|---|---|---|
| DSO Nano v3 | 6,500 | 1 MHz | No wireless dashboard; no mobile connectivity; single channel only |
| FNIRSI-138 | 2,500 | 200 kHz | No isolation; restricted to low-voltage DC signals; no current channel |
| JYE DSO150 | 2,000 | 200 kHz | No Wi-Fi; fixed onboard display; no AC mains sensing capability |
| Hantek DSO2C10 | 15,000 | 100 MHz | Expensive; no wireless; PC-tethered via USB |
| Owon HDS272S | 22,000 | 70 MHz | High bandwidth but very expensive; no power measurement |
| **OsciMobile (Ours)** | **2,000** | **5 kHz** | **AC/DC/Current multi-channel; wireless; web dashboard; CSV export; open-source** |

Table 6: Market comparison of similar oscilloscope products

**Key Differentiators of OsciMobile:**

- **Multi-channel sensing:** Simultaneous AC voltage, DC voltage, and current on one device.
- **Wireless & browser-based:** No drivers, no USB cable, no app install required.
- **Lowest cost** in its class for multi-channel, wireless operation.
- **Open-source firmware and dashboard** - extendable by the community.

## 11 Costing

| Cost Head | Amount |
|---|---|
| Bill of Materials (BOM) - Electronic Components | 1,915 |
| Veroboard, Mechanical Hardware, Connectors | 330 |
| Miscellaneous (Software, Labour etc.) | 200 |
| **Total Prototype Cost** | **2,445** |

Table 7: Detailed cost breakdown for OsciMobile prototype

**Projected Production Cost (100 units, PCB + enclosure):**

- Custom PCB manufacturing + assembly: ∼1,200/unit
- Enclosure (injection-moulded ABS): ∼300/unit
- Estimated production unit cost: ∼2,500–3,000/unit
- Suggested retail price: ∼4,000–5,000/unit

## 12 Marketing Strategy

### 12.1 Target Markets

1. **University Laboratory Kits:** The primary go-to-market strategy is to supply **OsciMobile as a lab kit** for Electrical Engineering departments at NITs, IITs, and engineering colleges across India. Each kit includes the hardware and access to the open-source firmware and OsciDash dashboard. Pricing at 3,500/unit (vs. 15,000+ for commercial handheld oscilloscopes) makes it highly attractive for bulk institutional procurement.

2. **Industrial IoT and MSME Sector:** Marketing as a low-cost power quality monitoring tool for small manufacturing units and MSME workshops through partnerships with industrial distributors.

### 12.2 Growth Strategy

- **Open Source Community:** Publish firmware and dashboard on GitHub to attract developer contributions, build credibility, and create organic marketing.
- **YouTube / Social Media:** Demo videos targeting the Indian electronics community (#ArduinoIndia, #ESP32Projects) for grassroots brand building.
- **Startup India / MSME Scheme:** Apply for MeitY startup grants and NIT Rourkela incubation support to fund the transition from prototype to product.

- **Product Differentiation:** Emphasize the unique combination of wireless connectivity, multi-channel sensing, and professional web UI - unavailable in any competing product at this price point.

### 12.3 Revenue Projections (Year 1)

| Segment | Units | Unit Price | Revenue |
|---|---|---|---|
| University Lab Kits | 200 | 3,500 | 7,00,000 |
| Online Retail | 150 | 4,000 | 6,00,000 |
| Industrial / MSME | 50 | 4,500 | 2,25,000 |
| **Total** | **400** | | **15,25,000** |